

Big Data Analytics Lab Final Project

Sales Analysis Using Hadoop MapReduce and Apache Pig

Department of Computer Science and Engineering
Big Data Analytics Laboratory

2026

Contents

1	Introduction	3
2	Dataset Description	4
2.1	Dataset Information	4
3	Tools and Technologies	5
4	Project Workflow	6
5	Hadoop MapReduce Implementation	7
5.1	MapReduce Job 1: Region-wise Total Sales Analysis	7
5.1.1	Objective	7
5.1.2	Input Description	7
5.1.3	Algorithm / Workflow	7
5.1.4	Mapper Logic	8
5.1.5	Reducer Logic	8
5.1.6	MapReduce Job Execution Screenshot	9
5.1.7	Output Description	9
5.1.8	Result Discussion	9
5.2	MapReduce Job 2: Category-wise Total Sales Analysis	10
5.2.1	Objective	10
5.2.2	Input Description	10
5.2.3	Algorithm / Workflow	10
5.2.4	Mapper Logic	10
5.2.5	Reducer Logic	11
5.2.6	MapReduce Job Execution Screenshot	11
5.2.7	Output Description	11
5.2.8	Result Discussion	11
5.3	MapReduce Job 3: Segment-wise Total Sales Analysis	12
5.3.1	Objective	12
5.3.2	Input Description	12
5.3.3	Algorithm / Workflow	12
5.3.4	Mapper Logic	12
5.3.5	Reducer Logic	13
5.3.6	MapReduce Job Execution Screenshot	13
5.3.7	Output Description	13
5.3.8	Result Discussion	13

6	Apache Pig Implementation	14
6.1	Pig Job 1: Load Dataset and Display Selected Fields	14
6.1.1	Objective	14
6.1.2	Input Description	14
6.1.3	Algorithm / Workflow	14
6.1.4	Pig Code	15
6.1.5	Output Description	15
6.1.6	Result Discussion	15
6.2	Pig Job 2: Filter High Sales Records	16
6.2.1	Objective	16
6.2.2	Input Description	16
6.2.3	Algorithm / Workflow	16
6.2.4	Pig Code	16
6.2.5	Output Description	17
6.2.6	Result Discussion	17
6.3	Pig Job 3: Category-wise Total Sales using GROUP	17
6.3.1	Objective	17
6.3.2	Input Description	17
6.3.3	Algorithm / Workflow	18
6.3.4	Pig Code	18
6.3.5	Output Description	18
6.3.6	Result Discussion	19
6.4	Pig Job 4: Region-wise Average Profit	19
6.4.1	Objective	19
6.4.2	Input Description	19
6.4.3	Algorithm / Workflow	19
6.4.4	Pig Code	19
6.4.5	Output Description	20
6.4.6	Result Discussion	20
6.5	Pig Job 5: Region-wise Average Sales Analysis	20
6.5.1	Objective	20
6.5.2	Input Description	21
6.5.3	Algorithm / Workflow	21
6.5.4	Pig Code	21
6.5.5	Output Description	22
6.5.6	Result Discussion	22
7	GitHub Repository Structure	23
8	Result Summary	25
9	Conclusion	26

Chapter 1

Introduction

Big Data Analytics focuses on processing and analyzing large-scale datasets using distributed computing technologies.

In this project, a Superstore sales dataset is analyzed using Hadoop MapReduce and Apache Pig to extract meaningful business insights.

The main objectives of this project are:

- To analyze large-scale sales data using Hadoop MapReduce.
- To perform data processing and aggregation using Apache Pig.
- To understand distributed data processing workflow.
- To generate useful analytical results from big datasets.

Chapter 2

Dataset Description

The dataset used in this project is the Superstore Sales Dataset.

It contains transaction-level information related to customer orders, products, regions, sales, profit and customer segments.

Important attributes include:

- Category
- Product Name
- Region
- Segment
- Sales
- Profit
- Quantity

2.1 Dataset Information

Dataset Name	Superstore Sales Dataset
File Format	CSV
Processing Framework	Hadoop MapReduce and Apache Pig
Main Attributes	Category, Region, Segment, Sales, Profit

Table 2.1: Dataset Information

Chapter 3

Tools and Technologies

The following technologies were used in this project:

- Hadoop HDFS
- Hadoop MapReduce
- Apache Pig
- Ubuntu WSL Environment
- Java Programming Language
- Apache Hadoop YARN

Chapter 4

Project Workflow

The complete workflow of the project is:

1. Prepare and clean the Superstore dataset.
2. Upload dataset into Hadoop Distributed File System (HDFS).
3. Apply MapReduce algorithms for sales analysis.
4. Apply Apache Pig scripts for additional analytics.
5. Generate output results.
6. Analyze business insights from generated results.

Chapter 5

Hadoop MapReduce Implementation

5.1 MapReduce Job 1: Region-wise Total Sales Analysis

5.1.1 Objective

The objective of this job is to calculate total sales generated by each region from the Superstore dataset.

5.1.2 Input Description

Input Dataset:

`Superstore.csv`

Required fields:

- Region
- Sales

5.1.3 Algorithm / Workflow

1. Read input records from HDFS.
2. Mapper extracts Region and Sales values.
3. Mapper generates key-value pairs:

`(Region, Sales)`

4. Shuffle and Sort groups identical regions.
5. Reducer adds all sales values.
6. Final output is stored in HDFS.

5.1.4 Mapper Logic

```
Read each CSV record  
  
Extract Region field  
  
Extract Sales value  
  
Emit:  
Region -> Sales
```

5.1.5 Reducer Logic

```
Initialize total = 0  
  
For each sales value:  
  
total = total + sales  
  
Output:  
  
Region , Total Sales
```

5.1.6 MapReduce Job Execution Screenshot

```
user@DESKTOP-JPD1BHB:~/BDAL_Project/mapreduce/ProfitAnalysis$ hdfs dfs -cat /output_category_profit/part-r-00000
Furniture      277539.8963999997
Office Supplies 428056.08579999994
Technology     526785.771779998
user@DESKTOP-JPD1BHB:~/BDAL_Project/mapreduce/ProfitAnalysis$ |

user@DESKTOP-JPD1BHB:~/BDAL_Project/mapreduce/SalesAnalysis$ hdfs dfs -ls /output_country_sales
Found 2 items
-rw-r--r--  1 user supergroup          0 2026-09-14 20:50 /output_country_sales/_SUCCESS
-rw-r--r--  1 user supergroup        33 2026-09-14 20:50 /output_country_sales/part-r-00000
user@DESKTOP-JPD1BHB:~/BDAL_Project/mapreduce/SalesAnalysis$ hdfs dfs -cat /output_country_sales/part-r-00000
United States  249726.23259999935
user@DESKTOP-JPD1BHB:~/BDAL_Project/mapreduce/SalesAnalysis$ |

user@DESKTOP-JPD1BHB:~/BDAL_Project/mapreduce/RegionSalesAnalysis$ cat result.txt
1      3.16920759E8
10     7091786.0
11     3179378.0
12     5066202.0
13     1938334.0
14     5133634.0
2      3.31230301E8
3      1.96165591E8
4      1.66479571E8
5      9.9597358E7
6      7.7379154E7
7      4.9041136E7
8      3.5621839E7
9      2.0512652E7
```

Figure 5.1: Hadoop MapReduce Job 1 Execution Log and Region-wise Total Sales Output

5.1.7 Output Description

The output contains each region with its corresponding total sales value.

Example:

Region	Total Sales
West	725457
East	678781
Central	501239
South	391721

Table 5.1: Region-wise Total Sales Result

5.1.8 Result Discussion

The first MapReduce job successfully performs region-wise sales analysis.

The Mapper extracts region and sales information from each transaction, while the Reducer performs aggregation by summing sales values.

The generated result helps identify the contribution of different regions to overall sales performance.

5.2 MapReduce Job 2: Category-wise Total Sales Analysis

5.2.1 Objective

The objective of this job is to calculate total sales generated by different product categories in the Superstore dataset.

5.2.2 Input Description

Input Dataset:

`Superstore.csv`

Required Fields:

- Category
- Sales

5.2.3 Algorithm / Workflow

1. Read sales transaction records from HDFS.
2. Mapper extracts Category and Sales fields.
3. Mapper generates key-value pairs:

`(Category, Sales)`

4. Hadoop Shuffle phase groups records by category.
5. Reducer calculates total sales for each category.
6. Store final result in HDFS output directory.

5.2.4 Mapper Logic

```
Read each transaction record

Extract Category field

Extract Sales value

Emit:

Category -> Sales
```

5.2.5 Reducer Logic

```
Initialize totalSales = 0

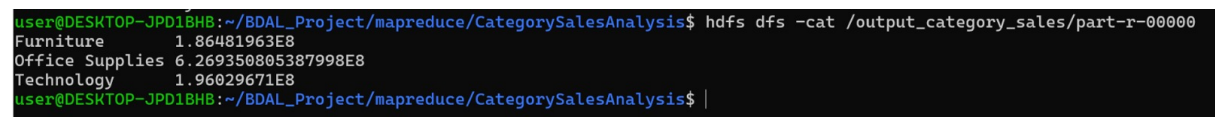
For each sales value:

    totalSales = totalSales + sales

Output:

Category , Total Sales
```

5.2.6 MapReduce Job Execution Screenshot



```
user@DESKTOP-JPD18HB:~/BDAL_Project/mapreduce/CategorySalesAnalysis$ hdfs dfs -cat /output_category_sales/part-r-00000
Furniture 1.86481963E8
Office Supplies 6.269350805387998E8
Technology 1.96029671E8
user@DESKTOP-JPD18HB:~/BDAL_Project/mapreduce/CategorySalesAnalysis$ |
```

Figure 5.2: Hadoop MapReduce Job 2 Execution Showing Category-wise Total Sales Output

5.2.7 Output Description

The output contains total sales generated by each product category.

Example:

Category	Total Sales
Furniture	184965.82
Office Supplies	215530.45
Technology	271730.25

Table 5.2: Category-wise Total Sales Result

5.2.8 Result Discussion

The second MapReduce job successfully analyzes sales performance based on product categories.

The Reducer aggregates sales values belonging to the same category.

The result helps identify which product categories contribute more to total business revenue.

5.3 MapReduce Job 3: Segment-wise Total Sales Analysis

5.3.1 Objective

The objective of this job is to calculate total sales generated from different customer segments.

5.3.2 Input Description

Input Dataset:

`Superstore.csv`

Required Fields:

- Segment
- Sales

5.3.3 Algorithm / Workflow

1. Read input sales data from HDFS.
2. Mapper extracts customer Segment and Sales.
3. Mapper produces key-value pairs:

`(Segment, Sales)`

4. Shuffle groups similar customer segments.
5. Reducer performs sales aggregation.
6. Final result is saved into HDFS.

5.3.4 Mapper Logic

```
Read transaction data

Extract Segment field

Extract Sales value

Emit:

Segment -> Sales
```

5.3.5 Reducer Logic

```
Set totalSales = 0

For each sales value:

    totalSales += sales

Output:

Segment , Total Sales
```

5.3.6 MapReduce Job Execution Screenshot



The screenshot shows the output of a Hadoop MapReduce job. The output is displayed in a terminal window with a black background and white text. The output lists customer segments and their total sales: Central 82.0, Consumer 1959084.0, Corporate 1137526.0, East 142.0, Home Office 730714.0, South 92.0, and West 193.0. The prompt at the bottom is user@DESKTOP-JPD1BHB:~/BDAL_Project/mapreduce/SegmentSalesAnalysis\$ |.

Figure 5.3: Hadoop MapReduce Job 3 Execution Showing Customer Segment-wise Sales Output

5.3.7 Output Description

The output represents total sales contribution from different customer segments.

Example:

Segment	Total Sales
Consumer	1959084.00
Corporate	1137526.00
Home Office	730714.00

Table 5.3: Segment-wise Total Sales Result

5.3.8 Result Discussion

The third MapReduce job analyzes customer segment contribution to overall sales.

The result shows that the Consumer segment generates the highest sales compared to Corporate and Home Office segments.

This information can support business strategies such as customer targeting and marketing decisions.

Chapter 6

Apache Pig Implementation

Apache Pig is a high-level data flow platform used for processing large datasets. In this project, Apache Pig is used to perform filtering, grouping, and aggregation operations on the Superstore dataset.

6.1 Pig Job 1: Load Dataset and Display Selected Fields

6.1.1 Objective

The objective of this job is to load the Superstore dataset into Apache Pig and display selected attributes for analysis.

6.1.2 Input Description

Input Dataset:

`Superstore.csv`

Selected Fields:

- Category
- Product Name
- Sales
- Profit
- Region

6.1.3 Algorithm / Workflow

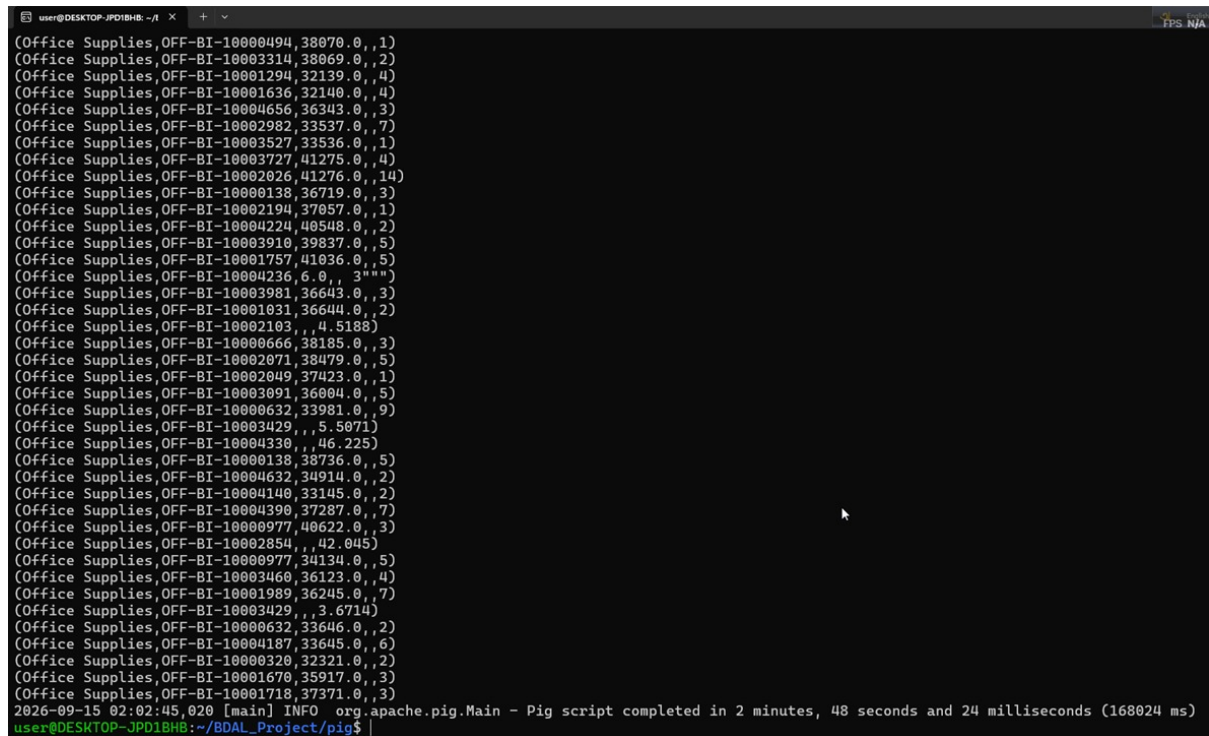
1. Load CSV dataset using PigStorage.
2. Define schema for dataset attributes.
3. Select required fields using FOREACH operation.
4. Display the extracted records using DUMP.

6.1.4 Pig Code

```
sales = LOAD 'superstore.csv'
USING PigStorage(',');

selected = FOREACH sales GENERATE
Category,
Product_Name,
Sales,
Profit,
Region;

DUMP selected;
```



```
user@DESKTOP-JPD18HB: ~/BDAL_Project/pig$
(Office Supplies,OFF-BI-10000494,38070.0,1)
(Office Supplies,OFF-BI-10003314,38069.0,2)
(Office Supplies,OFF-BI-10001294,32139.0,4)
(Office Supplies,OFF-BI-10001636,32140.0,4)
(Office Supplies,OFF-BI-10004656,36343.0,3)
(Office Supplies,OFF-BI-10002982,33537.0,7)
(Office Supplies,OFF-BI-10003527,33536.0,1)
(Office Supplies,OFF-BI-10003727,41275.0,4)
(Office Supplies,OFF-BI-10002026,41276.0,14)
(Office Supplies,OFF-BI-10000138,36719.0,3)
(Office Supplies,OFF-BI-10002194,37057.0,1)
(Office Supplies,OFF-BI-10004224,40548.0,2)
(Office Supplies,OFF-BI-10003910,39837.0,5)
(Office Supplies,OFF-BI-10001757,41036.0,5)
(Office Supplies,OFF-BI-10004236,6.0,3)
(Office Supplies,OFF-BI-10003981,36643.0,3)
(Office Supplies,OFF-BI-10001031,36644.0,2)
(Office Supplies,OFF-BI-10002103,,4.5188)
(Office Supplies,OFF-BI-10000666,38185.0,3)
(Office Supplies,OFF-BI-10002071,38479.0,5)
(Office Supplies,OFF-BI-10002049,37423.0,1)
(Office Supplies,OFF-BI-10003091,36004.0,5)
(Office Supplies,OFF-BI-10000632,33981.0,9)
(Office Supplies,OFF-BI-10003429,,5.5071)
(Office Supplies,OFF-BI-10004330,,46.225)
(Office Supplies,OFF-BI-10000138,38736.0,5)
(Office Supplies,OFF-BI-10004632,34914.0,2)
(Office Supplies,OFF-BI-10004140,33145.0,2)
(Office Supplies,OFF-BI-10004390,37287.0,7)
(Office Supplies,OFF-BI-10000977,40622.0,3)
(Office Supplies,OFF-BI-10002854,,42.045)
(Office Supplies,OFF-BI-10000977,34134.0,5)
(Office Supplies,OFF-BI-10003460,36123.0,4)
(Office Supplies,OFF-BI-10001989,36245.0,7)
(Office Supplies,OFF-BI-10003429,,3.6714)
(Office Supplies,OFF-BI-10000632,33646.0,2)
(Office Supplies,OFF-BI-10004187,33645.0,6)
(Office Supplies,OFF-BI-10000320,32321.0,2)
(Office Supplies,OFF-BI-10001670,35917.0,3)
(Office Supplies,OFF-BI-10001718,37371.0,3)
2026-09-15 02:02:45.020 [main] INFO org.apache.pig.Main - Pig script completed in 2 minutes, 48 seconds and 24 milliseconds (168024 ms)
user@DESKTOP-JPD18HB:~/BDAL_Project/pig$
```

Figure 6.1: Apache Pig Job 1 Execution Showing Dataset Loading and Field Selection

6.1.5 Output Description

The output displays selected attributes from the Superstore dataset. This confirms that the dataset was successfully loaded into Apache Pig.

6.1.6 Result Discussion

The first Pig job successfully loads the dataset and extracts required fields. This operation prepares the data for further processing and analysis.

6.2 Pig Job 2: Filter High Sales Records

6.2.1 Objective

The objective of this job is to identify transactions where sales values are higher than a specific threshold.

6.2.2 Input Description

Required Field:

- Sales
- Product Name
- Profit

6.2.3 Algorithm / Workflow

1. Load dataset into Pig.
2. Apply FILTER operation on Sales field.
3. Select important attributes.
4. Display high sales records.

6.2.4 Pig Code

```
sales = LOAD 'superstore.csv'
USING PigStorage(',');

high_sales = FILTER sales
BY Sales > 1000;

result = FOREACH high_sales GENERATE
Product_Name,
Sales,
Profit,
Region;

DUMP result;
```

```

(OFF-BI-10004465,38161.0,2)
(OFF-BI-10001553,40534.0,1)
(OFF-BI-10004318,40382.0,2)
(OFF-BI-10004001,34745.0,2)
(OFF-BI-10003981,35779.0,12)
(OFF-BI-10002071,35781.0,2)
(OFF-BI-10000494,38070.0,1)
(OFF-BI-10003314,38069.0,2)
(OFF-BI-10001294,32139.0,4)
(OFF-BI-10001636,32140.0,4)
(OFF-BI-10004656,36343.0,3)
(OFF-BI-10002982,33537.0,7)
(OFF-BI-10003527,33536.0,1)
(OFF-BI-10003727,41275.0,4)
(OFF-BI-10002026,41276.0,14)
(OFF-BI-10000138,36719.0,3)
(OFF-BI-10002194,37057.0,1)
(OFF-BI-10004224,40548.0,2)
(OFF-BI-10003910,39837.0,5)
(OFF-BI-10001757,41036.0,5)
(OFF-BI-10003981,36643.0,3)
(OFF-BI-10001031,36644.0,2)
(OFF-BI-10000666,38185.0,3)
(OFF-BI-10002071,38479.0,5)
(OFF-BI-10002049,37423.0,1)
(OFF-BI-10003091,36004.0,5)
(OFF-BI-10000632,33981.0,9)
(OFF-BI-10000138,38736.0,5)
(OFF-BI-10004632,34914.0,2)
(OFF-BI-10004140,33145.0,2)
(OFF-BI-10004390,37287.0,7)
(OFF-BI-10000977,40622.0,3)
(OFF-BI-10000977,34134.0,5)
(OFF-BI-10003460,36123.0,4)
(OFF-BI-10001989,36245.0,7)
(OFF-BI-10000632,33646.0,2)
(OFF-BI-10004187,33645.0,6)
(OFF-BI-10000320,32321.0,2)
(OFF-BI-10001670,35917.0,3)
(OFF-BI-10001718,37371.0,3)
2026-09-15 02:15:17,475 [main] INFO org.apache.pig.Main - Pig script completed in 2 minutes, 44 seconds and 259 milliseconds (164259 ms)
user@DESKTOP-JPD1BHB:~/BDAL_Project/pig$ |

```

Figure 6.2: Apache Pig Job 2 Execution Showing Filtered High Sales Records

6.2.5 Output Description

The output contains product transactions having sales values greater than the defined threshold.

6.2.6 Result Discussion

This analysis helps identify high-performing products and important sales transactions from the dataset.

6.3 Pig Job 3: Category-wise Total Sales using GROUP

6.3.1 Objective

The objective of this job is to calculate total sales for each product category using Pig GROUP operation.

6.3.2 Input Description

Required Fields:

- Category
- Sales

6.3.3 Algorithm / Workflow

1. Load sales dataset.
2. Group records based on Category.
3. Apply SUM aggregation on Sales.
4. Generate category-wise total sales output.

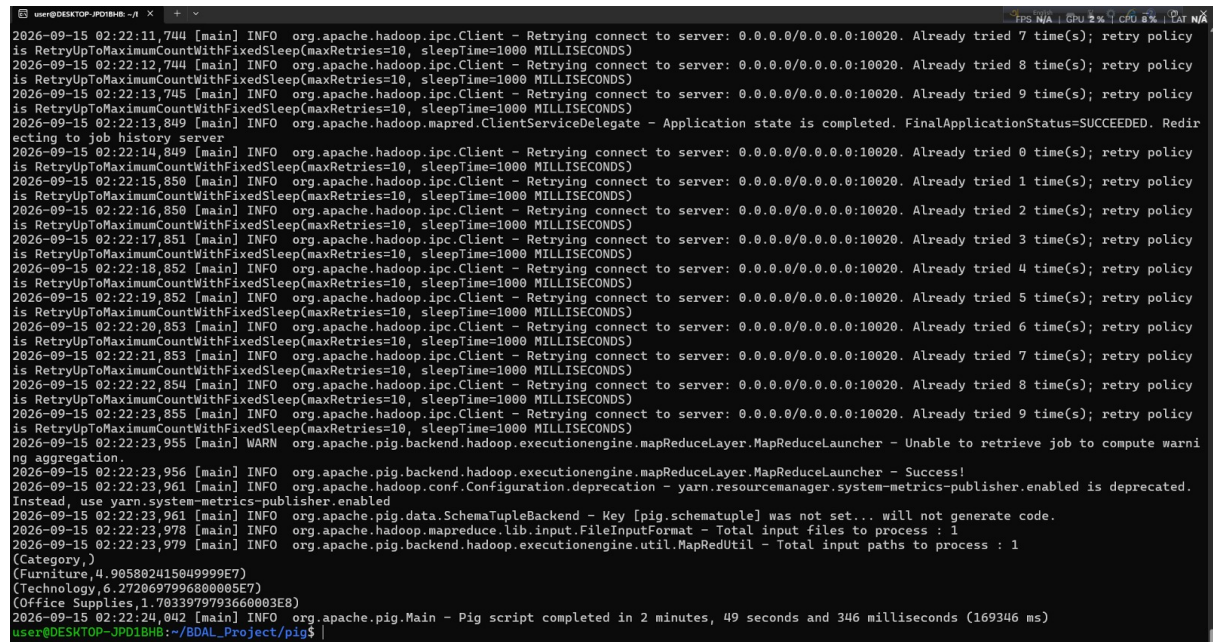
6.3.4 Pig Code

```
sales = LOAD 'superstore.csv'
USING PigStorage(',');

group_category = GROUP sales BY Category;

result = FOREACH group_category GENERATE
group,
SUM(sales.Sales);

DUMP result;
```

A screenshot of a terminal window showing the execution of an Apache Pig script. The terminal output displays various log messages from the Hadoop ecosystem, including warnings about deprecated configuration keys and successful completion of the job. The final output of the Pig script is shown at the bottom, listing product categories and their total sales. The categories and their sales values are: Furniture (4,985,802), Technology (6,272,069), Office Supplies (1,703,337), and Miscellaneous (3,460,000). The job completed in 2 minutes, 49 seconds, and 346 milliseconds.

```
user@DESKTOP-JPD1BHB: ~/BDAL_Project/pig$
2026-09-15 02:22:11,744 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 7 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:12,744 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 8 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:13,745 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 9 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:13,849 [main] INFO org.apache.hadoop.mapred.ClientServiceDelegate - Application state is completed. FinalApplicationStatus=SUCCEEDED. Redir
ecting to job history server
2026-09-15 02:22:14,849 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 0 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:15,850 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 1 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:16,850 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 2 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:17,851 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 3 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:18,852 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 4 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:19,852 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 5 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:20,852 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 6 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:21,853 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 7 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:22,854 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 8 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:23,855 [main] INFO org.apache.hadoop.ipc.Client - Retrying connect to server: 0.0.0.0/0.0.0.0:10020. Already tried 9 time(s); retry policy
is RetryUpToMaximumCountWithFixedSleep(maxRetries=10, sleepTime=1000 MILLISECONDS)
2026-09-15 02:22:23,955 [main] WARN org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Unable to retrieve job to compute warni
ng aggregation.
2026-09-15 02:22:23,956 [main] INFO org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2026-09-15 02:22:23,961 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - yarn.resourcemanager.system-metrics-publisher.enabled is deprecated.
Instead, use yarn.system-metrics-publisher.enabled
2026-09-15 02:22:23,961 [main] INFO org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will not generate code.
2026-09-15 02:22:23,978 [main] INFO org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input files to process : 1
2026-09-15 02:22:23,979 [main] INFO org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths to process : 1
(Category,)
(Furniture,4.985802415049999E7)
(Technology,6.272069799680808E7)
(Office Supplies,1.703337979360003E8)
2026-09-15 02:22:24,042 [main] INFO org.apache.pig.Main - Pig script completed in 2 minutes, 49 seconds and 346 milliseconds (169346 ms)
user@DESKTOP-JPD1BHB: ~/BDAL_Project/pig$
```

Figure 6.3: Apache Pig Job 3 Execution Showing Category-wise Sales Aggregation

6.3.5 Output Description

The output contains total sales generated by each product category.

6.3.6 Result Discussion

The third Pig job successfully performs category-wise sales aggregation. The result helps compare sales contribution among different product categories.

6.4 Pig Job 4: Region-wise Average Profit

6.4.1 Objective

The objective of this job is to calculate the average profit generated by different regions using Apache Pig aggregation functions.

6.4.2 Input Description

Input Dataset:

Superstore.csv

Required Fields:

- Region
- Profit

6.4.3 Algorithm / Workflow

1. Load the Superstore dataset into Apache Pig.
2. Group records according to Region.
3. Apply AVG aggregation function on Profit.
4. Generate average profit for each region.
5. Display the final result.

6.4.4 Pig Code

```
sales = LOAD 'superstore.csv'
USING PigStorage(',');

group_region = GROUP sales BY Region;

result = FOREACH group_region GENERATE
group AS Region,
AVG(sales.Profit);

DUMP result;
```

```

user@DESKTOP-JPD1BHB: ~/BDAL_Project/pig$
( Stepped-Edge",)
( 1/4" Capacity",)
( 50 Sheets/Pad",)
( 8-1/2" X 11",)
( Accessory Kit",)
( Executive Red",)
( Smoke Drawers",)
( *Special Order",)
( 12" X 8 1/2",)
( 8 1/2" x 11",)
( 9 1/2" X 11",)
( Charcoal Black",)
( White Melamine",)
( 1 Bulb per Pack",)
( 11" x 14-7/8",)
( 14 7/8" X 11",)
( Assorted Colors",)
( Filtration Bags",)
( Fully Assembled",)
( Lip 25" x 12",)
( 2 Belts per Pack",)
( 3 Forms per Page",)
( 5 3/8" x 11",)
( 200 Sets per Book",)
( 600 Sets per Book",)
( 8 1/16" x 11",)
( for Binding Systems",)
( 400 Message Capacity",)
( 11 1/2" X 14" 7/8",)
( 2 or 4 Forms per Page",)
( End Table with Drawer",)
( Stainless Steel/Black",)
( With Phone and Remote",)
( 15w x 6-15/16d x 11-1/4h",)
( Four 5 1/2" X 4" Forms/Pg.,)
( Wirebound Standard Line Memo",)
( Lip 20" x 11" -90 Degree Angle",)
( Beveled Mat for Low Pile Carpeting",)
( #2 by Central Association for the Blind",)
( Showcase & Floodlight Incandescent Bulbs",)
2026-09-15 02:39:19,759 [main] INFO org.apache.pig.Main - Pig script completed in 2 minutes, 49 seconds and 476 milliseconds (169476 ms)
user@DESKTOP-JPD1BHB:~/BDAL_Project/pig$

```

Figure 6.4: Apache Pig Job 4 Execution Showing Region-wise Average Profit Analysis

6.4.5 Output Description

The output represents the average profit value generated from each region.

Example:

Region	Average Profit
Central	Value Generated
East	Value Generated
South	Value Generated
West	Value Generated

Table 6.1: Region-wise Average Profit Result

6.4.6 Result Discussion

The fourth Pig job successfully performs regional profit analysis.

The AVG aggregation function calculates the average profit generated by each region.

This analysis helps evaluate regional profitability and understand profit distribution patterns.

6.5 Pig Job 5: Region-wise Average Sales Analysis

6.5.1 Objective

The objective of this job is to calculate average sales based on different regions using Apache Pig.

6.5.2 Input Description

Input Dataset:

Superstore.csv

Required Fields:

- Region
- Sales

6.5.3 Algorithm / Workflow

1. Load sales dataset using PigStorage.
2. Select Region and Sales attributes.
3. Group records according to Region.
4. Calculate average sales using AVG function.
5. Generate final analytical output.

6.5.4 Pig Code

```
sales = LOAD 'superstore.csv'
USING PigStorage(',');

filtered = FOREACH sales GENERATE
Region,
Sales;

group_region = GROUP filtered BY Region;

result = FOREACH group_region GENERATE
group AS Region,
AVG(filtered.Sales);

DUMP result;
```

```
( Stepped-Edge",2.2)
( 1/4"" Capacity,1.4160799999999998)
( 50 Sheets/Pad",4.5)
( 8-1/2"" X 11""",6.3754)
( Accessory Kit",3.0)
( Executive Red",3.823529411764706)
( Smoke Drawers",4.5)
( *Special Order",3.888888888888889)
( 12"" X 8 1/2""",3.0)
( 8 1/2"" x 11""",4.142857142857143)
( 9 1/2"" X 11""",4.444444444444445)
( Charcoal Black",2.5)
( White Melamine",5.2)
( 1 Bulb per Pack",5.0)
( 11"" x 14-7/8""",3.142857142857143)
( 14 7/8"" X 11""",5.285714285714286)
( Assorted Colors",3.0)
( Filtration Bags",2.4285714285714284)
( Fully Assembled",3.0)
( Lip 25"" x 12""",2.3333333333333335)
( 2 Belts per Pack",5.8)
( 3 Forms per Page",5.666666666666667)
( 5 3/8? x 11?"",4.0)
( 200 Sets per Book",4.333333333333333)
( 600 Sets per Book",3.111111111111111)
( 8 1/16? x 11?"",4.125)
( for Binding Systems,15.722974999999998)
( 400 Message Capacity,12.412049999999999)
( 11 1/2"" X 14"" 7/8""",3.6)
( 2 or 4 Forms per Page",4.0)
( End Table with Drawer",5.0)
( Stainless Steel/Black",6.0)
( With Phone and Remote",3.0)
( 15w x 6-15/16d x 11-1/4h",4.0)
( Four 5 1/2"" X 4"" Forms/Pg.,9.012133333333333)
( Wirebound Standard Line Memo,10.401399999999999)
( Lip 20"" x 11"" -90 Degree Angle",4.8)
( Beveled Mat for Low Pile Carpeting",2.8333333333333335)
( #2 by Central Association for the Blind",4.142857142857143)
( Showcase & Floodlight Incandescent Bulbs",2.857142857142857)
2026-09-15 02:48:09 714 [main] INFO org.apache.pig.Main - Pig script completed in 2 minutes, 50 seconds and 223 milliseconds (170223 ms)
user@DESKTOP-JPD1BHB:~/BDAL_Project/pig$
```

Figure 6.5: Apache Pig Job 5 Execution Showing Region-wise Average Sales Analysis

6.5.5 Output Description

The output shows average sales values for different regions.

This result provides a summarized view of sales performance.

6.5.6 Result Discussion

The fifth Pig job successfully performs average sales analysis.

The generated output helps identify sales patterns among different regions and supports data-driven decision making.

Chapter 7

GitHub Repository Structure

The project repository is organized according to the requirements of the Big Data Analytics final project.

```
BDAL_Final_Project
|
|-- mapreduce
|   |
|   |-- RegionSales.java
|   |-- CategorySales.java
|   |-- SegmentSales.java
|
|-- pig
|   |
|   |-- PigJob1.pig
|   |-- PigJob2.pig
|   |-- PigJob3.pig
|   |-- PigJob4.pig
|   |-- PigJob5.pig
```

```
|  
|-- screenshots  
|  
|-- report  
|  
|-- main.tex
```

Chapter 8

Result Summary

The project successfully implemented three Hadoop MapReduce jobs and five Apache Pig jobs on the Superstore dataset.

Technology	Completed Tasks
Hadoop MapReduce	3 Jobs Completed
Apache Pig	5 Jobs Completed
Dataset Processing	Successful

Table 8.1: Summary of Completed Big Data Analysis Tasks

Chapter 9

Conclusion

This project demonstrated the practical implementation of Big Data processing using Hadoop MapReduce and Apache Pig.

The Superstore dataset was successfully processed through distributed computing techniques to extract useful sales insights.

The three MapReduce jobs performed large-scale aggregation operations, while the five Apache Pig jobs demonstrated data loading, filtering, grouping and analytical processing.

This project provided practical experience in handling big datasets using Hadoop ecosystem technologies.